Intel SGX Enclave 생성 과정의 코드 흐름 분석

개요

이 문서는 Intel SGX SDK에서 `sgx_create_enclave()` 함수부터 시작하여 전체 Enclave 생성 과정을 실제 소스 코드를 따라가면서 단계별로 분석한 결과입니다. URTS의 enclave.cpp에서 시작하여 loader.cpp의 메모리 할당 및 페이지 설정 로직, Launch Control 검증 과정, EINIT 명령어 실행까지의 전체 코드 경로를 추적합니다.

1. sgx_create_enclave() 함수 진입점 분석

1.1 함수 호출 계층 구조

```
// psw/urts/linux/urts.cpp
extern "C" sgx_status_t sgx_create_enclave(const char *file_name,
const int debug,
sgx_launch_token_t *launch_token,
int *launch_token_updated,
sgx_enclave_id_t *enclave_id,
sgx_misc_attribute_t *misc_attr)
{
return __sgx_create_enclave_ex(file_name, debug, launch_token,
launch_token_updated, enclave_id,
misc_attr, 0, NULL);
```

```
}
```

분석 포인트:

- `sgx_create_enclave()` 는 단순한 래퍼 함수로, 내부적으로 `__sgx_create_enclave_ex()` 를 호출
- 확장 기능 매개변수(`ex_features` , `ex_features_p`)는 0과 NULL로 설정되어 기본 생성 모드로 동작

1.2 __sgx_create_enclave_ex() 핵심 구현

```
// psw/urts/linux/urts.cpp
extern "C" sgx_status_t __sgx_create_enclave_ex(const char *file_name,
const int debug,
sgx_launch_token_t *launch_token,
int *launch_token_updated,
sgx_enclave_id_t *enclave_id,
sgx_misc_attribute_t *misc_attr,
const uint32_t ex_features,
const void* ex_features_p[32])
{
sgx_status_t ret = SGX_SUCCESS;


// 1. 매개변수 검증
if(TRUE != debug && FALSE != debug)
return SGX_ERROR_INVALID_PARAMETER;
```

```
if (!_check_ex_params_(ex_features, ex_features_p))
return SGX_ERROR_INVALID_PARAMETER;

// 2. Enclave 파일 열기
int fd = open(file_name, O_RDONLY);
if(-1 == fd) {
SE_TRACE(SE_TRACE_ERROR, "Couldn't open the enclave file, error = %d\n", errno);
return SGX_ERROR_ENCLAVE_FILE_ACCESS;
}

// 3. 파일 경로 정규화
se_file_t file = {NULL, 0, false};
char  resolved_path[PATH_MAX] = {0};
file.name = realpath(file_name, resolved_path);
file.name_len = (uint32_t)strnlen_s(resolved_path, PATH_MAX);

// 4. 실제 Enclave 생성 호출
ret = _create_enclave_ex(!!debug, fd, file, NULL, launch_token,
launch_token_updated, enclave_id, misc_attr,
ex_features, ex_features_p);

// 5. 플랫폼 능력 정보 설정
```

```
if(SGX_SUCCESS != ret && misc_attr) {
sgx_misc_attribute_t plat_cap;
memset(&plat_cap, 0, sizeof(plat_cap));
get_enclave_creator()->get_plat_cap(&plat_cap);
memcpy_s(misc_attr, sizeof(sgx_misc_attribute_t), &plat_cap,
sizeof(sgx_misc_attribute_t));
}

close(fd);
return ret;
}
```

분석 포인트:

- 파일 접근 권한 검사 및 파일 디스크립터 생성
- `realpath()` 를 통한 절대 경로 변환으로 심볼릭 링크 해결
- `_create_enclave_ex()` 호출로 실제 생성 로직 위임
- 실패 시 플랫폼 능력 정보를 `misc_attr` 에 설정하여 호환성 정보 제공

2. _create_enclave_ex() 핵심 생성 로직

2.1 파일 매핑 및 파서 초기화

```
// psw/urts/urts_com.h
sgx_status_t  _create_enclave_ex(const  bool  debug, se_file_handle_t  pfile,
```

```
se_file_t&  file, le_prd_css_file_t  *prd_css_file,
sgx_launch_token_t  *launch, int  *launch_updated,
sgx_enclave_id_t  *enclave_id,
sgx_misc_attribute_t  *misc_attr,
const  uint32_t  ex_features,
const  void*  ex_features_p[32])
{
unsigned  int ret = SGX_SUCCESS;
off_t file_size = 0;
map_handle_t* mh = NULL;


// 1. 파일 메모리 매핑
mh = map_file(pfile, &file_size);
if (!mh)
return SGX_ERROR_OUT_OF_MEMORY;


// 2. 버퍼 기반 생성 함수 호출
ret = _create_enclave_from_buffer_ex(debug, mh->base_addr, (uint64_t)(file_size)
file, prd_css_file, enclave_id, misc_attr,
ex_features, ex_features_p);


unmap_file(mh);
return (sgx_status_t)ret;
```

```
}
```

분석 포인트:

- `map_file()` 을 통해 Enclave 바이너리를 메모리에 매핑
- 파일 크기 정보를 `off_t` 타입으로 획득
- `_create_enclave_from_buffer_ex()` 로 실제 처리 위임하여 파일/버퍼 처리 통합

2.2 _create_enclave_from_buffer_ex() 상세 분석

```
// psw/urts/urts_com.h
sgx_status_t _create_enclave_from_buffer_ex(const bool debug, uint8_t *base_
uint64_t file_size, se_file_t& file,
le_prd_css_file_t *prd_css_file,
sgx_enclave_id_t *enclave_id,
sgx_misc_attribute_t *misc_attr,
const uint32_t ex_features,
const void* ex_features_p[32])
{
unsigned int ret = SGX_SUCCESS;
sgx_misc_attribute_t sgx_misc_attr;
metadata_t *metadata = NULL;
SGXLaunchToken *lc = NULL;
memset(&sgx_misc_attr, 0, sizeof(sgx_misc_attribute_t));
```

```
// 1. 매개변수 검증
if(NULL == base_addr || NULL == enclave_id)
return SGX_ERROR_INVALID_PARAMETER;


#ifndef SE_SIM
// 2. 플랫폼 검증
ret = validate_platform();
if(ret != SGX_SUCCESS)
return (sgx_status_t)ret;
#endif


// 3. ELF 파서 초기화
PARSER parser(base_addr, file_size);
if(SGX_SUCCESS != (ret = parser.run_parser())) {
goto clean_return;
}


// 4. HW/시뮬레이션 모드 호환성 검사
if(get_enclave_creator()->use_se_hw() != (!parser.get_symbol_rva("g_global_data_
SE_TRACE_WARNING("HW and Simulation mode incompatibility detected.\n");
ret = SGX_ERROR_MODE_INCOMPATIBLE;
goto clean_return;
}
```

```
// 5. 메타데이터 추출 및 검증
if(SGX_SUCCESS != (ret = get_metadata(&parser, debug, &metadata, &sgx_misc_attr)
goto clean_return;
}

// 6. Launch Token 생성
lc = new  SGXLaunchToken(&metadata->enclave_css, &sgx_misc_attr.secs_attr, NULL)

// 7. XSAVE 정보 초기화
init_xsave_info();

// 8. 실제 Enclave 생성
do {
ret = __create_enclave(parser, base_addr, metadata, file, debug, lc,
prd_css_file, enclave_id, misc_attr,
ex_features, ex_features_p);
} while(SGX_ERROR_ENCLAVE_LOST == ret); // 전력 전환 중 재시도

// 9. 에러 코드 변환
if(SE_ERROR_INVALID_LAUNCH_TOKEN == ret)
ret = SGX_ERROR_INVALID_LAUNCH_TOKEN;
if(SE_ERROR_INVALID_MEASUREMENT == ret)
```

```
ret = SGX_ERROR_INVALID_SIGNATURE;

if (SE_ERROR_INVALID_ISVSVNLE == ret)

ret = SGX_ERROR_UPDATE_NEEDED;



clean_return:

if(lc != NULL)

delete lc;

return (sgx_status_t)ret;

}
```

분석 포인트:

- ELF 파서를 통한 바이너리 구조 분석
- HW/시뮬레이션 모드 호환성 검사로 잘못된 라이브러리 링크 방지
- 메타데이터 추출 및 버전 호환성 검증
- Launch Token 생성으로 Launch Control 준비
- 전력 전환으로 인한 실패 시 자동 재시도 메커니즘

3. URTS의 Enclave 로더 (CLoader) 분석

3.1 __create_enclave() 함수의 CLoader 초기화

```
// psw/urts/urts_com.h

static  int  __create_enclave(BinParser  &parser,

uint8_t*  base_addr,
```

```
const metadata_t *metadata,
se_file_t& file,
const bool debug,
SGXLaunchToken *lc,
le_prd_css_file_t *prd_css_file,
sgx_enclave_id_t *enclave_id,
sgx_misc_attribute_t *misc_attr,
const uint32_t ex_features,
const void* ex_features_p[32])
{
debug_enclave_info_t *debug_info = NULL;
int ret = SGX_SUCCESS;
sgx_config_id_t *config_id = NULL;
sgx_config_svn_t config_svn = 0;
sgx_kss_config_t *kss_config = NULL;
sgx_uswitchless_config_t* us_config = NULL;
// 1. CLoader 객체 생성
CLoader loader(base_addr, parser);


// 2. KSS(Key Separation & Sharing) 설정 추출
if (get_ex_feature_pointer(SGX_CREATE_ENCLAVE_EX_KSS, ex_features,
ex_features_p, (void **)&kss_config) == -1)
return SGX_ERROR_INVALID_PARAMETER;
```

```
if (kss_config) {
config_id = &(kss_config->config_id);
config_svn = kss_config->config_svn;
}

// 3. Enclave 로딩 실행
ret = loader.load_enclave_ex(lc, debug, metadata, config_id, config_svn,
prd_css_file, misc_attr);
if (ret != SGX_SUCCESS) {
return ret;
}

// 4. CEnclave 객체 생성 및 초기화
CEnclave* enclave = new  CEnclave();
uint32_t enclave_version = SDK_VERSION_1_5;
uint64_t urts_version = META_DATA_MAKE_VERSION(MAJOR_VERSION,MINOR_VERSION);
if (MAJOR_VERSION_OF_METADATA(metadata->version) % SGX_MAJOR_VERSION_GAP ==
MAJOR_VERSION_OF_METADATA(urts_version)% SGX_MAJOR_VERSION_GAP) {
enclave_version = SDK_VERSION_3_0;
}

ret = enclave->initialize(file, loader, metadata->enclave_size,
metadata->tcs_policy, enclave_version,
```

```
metadata->tcs_min_pool);
if (ret != SGX_SUCCESS) {
loader.destroy_enclave();
delete enclave;
return ret;
}

// ... 추가 초기화 로직
}
```

분석 포인트:

- `CLoader` 객체는 매핑된 파일 베이스 주소와 파서를 받아 초기화
- KSS 기능 지원 시 설정 ID와 보안 버전 추출
- `load_enclave_ex()` 호출로 실제 로딩 작업 수행
- SDK 버전에 따른 Enclave 버전 결정 로직

3.2 CLoader::load_enclave_ex() 구현 분석

```
// psw/urts/loader.h
class CLoader: private Uncopyable
{
public:
CLoader(uint8_t *mapped_file_base, BinParser &parser);
virtual ~CLoader();
```

```
int load_enclave_ex(SGXLaunchToken *lc, bool is_debug,
const metadata_t *metadata, sgx_config_id_t *config_id,
sgx_config_svn_t config_svn,
le_prd_css_file_t *prd_css_file = NULL,
sgx_misc_attribute_t *misc_attr = NULL);


private:
int build_image(SGXLaunchToken * const lc, sgx_attributes_t * const secs_
sgx_config_id_t *config_id, sgx_config_svn_t config_svn,
le_prd_css_file_t *prd_css_file,
sgx_misc_attribute_t * const misc_attr);
int build_secs(sgx_attributes_t * const secs_attr, sgx_config_id_t *config_
sgx_config_svn_t config_svn, sgx_misc_attribute_t * const misc_attr);
int build_contexts(layout_t *layout_start, layout_t *layout_end, uint64_t de
int build_pages(const uint64_t start_rva, const uint64_t size,
const void *source, const sec_info_t &sinfo, const uint32_t attr);


uint8_t *m_mapped_file_base; // 매핑된 파일 베이스 주소
sgx_enclave_id_t m_enclave_id; // Enclave ID
void *m_start_addr; // Enclave 시작 주소
uint64_t m_elrange_start_address; // EL Range 시작 주소
uint64_t m_elrange_size; // EL Range 크기
std::vector<std::pair<tcs_t *, bool>> m_tcs_list; // TCS 리스트
```

```
const metadata_t *m_metadata; // 메타데이터
secs_t m_secs; // SECS 구조체
BinParser &m_parser; // 바이너리 파서
};
```

분석 포인트:

- `m_mapped_file_base` : ELF 바이너리가 매핑된 메모리 주소
- `m_start_addr` : 생성된 Enclave의 가상 메모리 시작 주소
- `m_tcs_list` : Thread Control Structure 목록 관리
- `m_secs` : SGX Enclave Control Structure 저장

4. 메모리 할당 및 페이지 설정 로직 분석

4.1 build_mem_region() 페이지별 메모리 구성

```
// psw/urts/loader.cpp
int CLoader::build_mem_region(const section_info_t &sec_info)
{
int ret = SGX_SUCCESS;
uint64_t offset = 0;
sec_info_t sinfo;
memset(&sinfo, 0, sizeof(sinfo));

// 초기화된 데이터를 포함하는 섹션의 페이지들을 구성
```

```
// 각 페이지는 재배치 데이터를 포함할 수 있으므로 개별적으로 추가 필요
while(offset < sec_info.raw_data_size) {
uint64_t rva = sec_info.rva + offset;
uint64_t size = MIN((SE_PAGE_SIZE - PAGE_OFFSET(rva)),
(sec_info.raw_data_size - offset));
sinfo.flags = sec_info.flag;


// 재배치 페이지이면서 쓰기 불가능한 경우 쓰기 권한 추가
if(is_relocation_page(rva, sec_info.bitmap) && !(sec_info.flag & SI_FLAG_W)) {
sinfo.flags = sec_info.flag | SI_FLAG_W;
assert(g_enclave_creator != NULL);
if(g_enclave_creator->use_se_hw() == true) {
ret = mprotect((void*)(TRIM_TO_PAGE(rva) + (uint64_t)m_start_addr),
SE_PAGE_SIZE,
(int)(sinfo.flags & SI_MASK_MEM_ATTRIBUTE));
if(ret != 0) {
SE_TRACE(SE_TRACE_WARNING,
"mprotect(rva=0x%llx, len=%d, flags=%d) failed\n",
rva, SE_PAGE_SIZE, int(sinfo.flags & SI_MASK_MEM_ATTRIBUTE));
return SGX_ERROR_UNEXPECTED;
}
}
}
```

```
// 페이지 크기에 따른 빌드 방식 선택
if (size == SE_PAGE_SIZE)
ret = build_pages(rva, size, sec_info.raw_data + offset, sinfo, ADD_EXTEND_PAGE)
else
ret = build_partial_page(rva, size, sec_info.raw_data + offset, sinfo, ADD_EXTEN

if(SGX_SUCCESS != ret)
break;
offset += size;
}

return ret;
}
```

분석 포인트:

- 페이지 단위(4KB)로 메모리 영역 구성
- 재배치 정보가 있는 페이지는 임시로 쓰기 권한 부여
- `mprotect()` 시스템 콜을 통한 메모리 보호 속성 설정
- 완전한 페이지와 부분 페이지에 대한 다른 처리 방식

4.2 is_relocation_page() 재배치 페이지 검사

```
// psw/urts/loader.cpp
```

```
bool CLoader::is_relocation_page(const uint64_t rva, std::vector<uint8_t> *bi
{
uint64_t page_frame = rva >> SE_PAGE_SHIFT; // 페이지 프레임 번호 계산
size_t index = (size_t)(page_frame / 8); // 비트맵 인덱스
if(bitmap && (index < bitmap->size())) {
return ((*bitmap)[index] & (1 << (page_frame % 8))); // 비트 검사
}
return false;
}
```

분석 포인트:

- RVA(Relative Virtual Address)를 페이지 프레임 번호로 변환
- 비트맵을 통한 효율적인 재배치 페이지 식별
- 8비트 단위로 압축된 비트맵 구조 사용

5. Launch Control 검증 과정 분석

5.1 SGXLaunchToken 클래스 구조

```
// Launch Token 생성 (psw/urts/urts_com.h)
lc = new SGXLaunchToken(&metadata->enclave_css, &sgx_misc_attr.secs_attr, NULL)
```

분석 포인트:

- `enclave_css` : Enclave의 CSS(Code Signing Structure) 정보

- `secs_attr` : SECS 속성 정보
- Launch Token은 Enclave의 서명 정보와 속성을 기반으로 생성

5.2 Launch Enclave와의 상호작용 코드 추적

Launch Control 과정은 다음과 같은 단계로 진행됩니다:

1. **Launch Token 검증**: 기존 토큰의 유효성 검사
2. **Launch Enclave 호출**: 필요시 새로운 토큰 생성 요청
3. **토큰 업데이트**: 새로 생성된 토큰으로 업데이트

```
// Launch Control 검증 과정의 핵심 로직
int validate_launch_token(SGXLaunchToken *lc, const enclave_css_t *enclave_c
const sgx_attributes_t *secs_attr)
{
// 1. 토큰 유효성 검사
if (!lc->is_valid()) {
// 2. Launch Enclave를 통한 새 토큰 생성
return request_new_token_from_le(enclave_css, secs_attr);
}
// 3. 토큰과 Enclave 정보 일치성 검사
if (!lc->match_enclave(enclave_css, secs_attr)) {
return request_token_update(enclave_css, secs_attr);
}
return SGX_SUCCESS;
```

```
}
```

분석 포인트:

- Launch Token의 유효성은 암호학적 서명으로 검증
- Enclave CSS와 토큰 정보의 일치성 검사
- 불일치 시 Launch Enclave를 통한 새 토큰 요청

6. EINIT 명령어 실행 과정 분석

6.1 EnclaveCreatorHW::try_init_enclave() 구현

```
// psw/urts/linux/enclave_creator_hw.cpp
int  EnclaveCreatorHW::try_init_enclave(sgx_enclave_id_t  enclave_id,
enclave_css_t  *enclave_css,
token_t  *launch)
{
UNUSED(launch);


// 1. EINIT 구조체 준비
enclave_init_sgx_t enclave_init_sgx = {0};
if (0 != memcpy_s(enclave_init_sgx.sigstruct, SIGSTRUCT_SIZE,
enclave_css, SIGSTRUCT_SIZE))
return SGX_ERROR_UNEXPECTED;
```

```
// 2. EINIT 명령어 실행

uint32_t enclave_error = ENCLAVE_ERROR_SUCCESS;

enclave_initialize((void*)enclave_id, &enclave_init_sgx,

sizeof(enclave_init_sgx), &enclave_error);



if (enclave_error)

return  error_api2urts(enclave_error);



// 3. 신호 핸들러 등록

se_mutex_lock(&m_sig_mutex);

if(false == m_sig_registered) {

reg_sig_handler();

m_sig_registered = true;

}

se_mutex_unlock(&m_sig_mutex);



return SGX_SUCCESS;

}
```

분석 포인트:

- `enclave_css` 를 `sigstruct` 형태로 변환하여 EINIT 구조체 준비
- `enclave_initialize()` 함수를 통한 실제 EINIT 명령어 실행
- EINIT 성공 후 신호 핸들러 등록으로 예외 처리 준비

6.2 하드웨어 명령어 래퍼 분석

EINIT 명령어는 다음과 같은 계층을 통해 실행됩니다:

```
Application Layer
↓
EnclaveCreatorHW::try_init_enclave()
↓
enclave_initialize() [Enclave Common API]
↓
SGX Driver (ioctl interface)
↓
EINIT Hardware Instruction
```

시뮬레이션 모드에서의 EINIT 구현:

```
// sdk/simulation/uinst/u_instructions.cpp
uintptr_t  _EINIT(secs_t*  secs, enclave_css_t  *css, token_t  *launch)
{
CEnclaveMngr *mngr = CEnclaveMngr::get_instance();
// 1. Enclave 관리자에서 SECS 검색
CEnclaveSim *ce = mngr->get_enclave(secs);
if (ce == NULL) {
return SGX_ERROR_INVALID_ENCLAVE_ID;
}
```

```
// 2. 시뮬레이션 모드 EINIT 실행
return (int)DoEINIT_SW(ce->get_secs(), css, launch);
}
```

분석 포인트:

- 하드웨어 모드: SGX 드라이버를 통한 실제 EINIT 명령어 실행
- 시뮬레이션 모드: 소프트웨어 에뮬레이션을 통한 EINIT 동작 구현
- 두 모드 모두 동일한 인터페이스를 통해 투명하게 처리

7. 전체 코드 흐름 요약

7.1 함수 호출 순서

```
1. sgx_create_enclave()
↓
2. __sgx_create_enclave_ex()
↓
3. _create_enclave_ex()
↓
4. _create_enclave_from_buffer_ex()
↓
5. __create_enclave()
↓
6. CLoader::load_enclave_ex()
```

```
↓

7. CLoader::build_image()

↓

8. EnclaveCreatorHW::create_enclave()

↓

9. EnclaveCreatorHW::try_init_enclave()

↓

10. enclave_initialize() [EINIT 실행]
```

7.2 주요 데이터 구조 변환

```
ELF Binary (File)

↓ [ElfParser]

metadata_t (Enclave Metadata)

↓ [CLoader]

secs_t (SGX Enclave Control Structure)

↓ [EnclaveCreator]

SGX Hardware Enclave
```

7.3 메모리 관리 과정

1. **파일 매핑**: `map_file()` 을 통한 ELF 바이너리 메모리 매핑
2. **가상 메모리 할당**: `enclave_create_ex()` 를 통한 Enclave 가상 주소 공간 할당

3. **페이지별 구성**: `build_pages()` 를 통한 4KB 단위 페이지 구성

4. **메모리 보호**: `mprotect()` 를 통한 페이지별 권한 설정

5. **EINIT 실행**: 하드웨어를 통한 Enclave 활성화

8. 결론

Intel SGX Enclave 생성 과정은 다음과 같은 핵심 단계로 구성됩니다:

1. **파일 처리**: ELF 바이너리 파싱 및 메타데이터 추출

2. **메모리 할당**: 가상 주소 공간 할당 및 페이지별 구성

3. **Launch Control**: Launch Token을 통한 허가 검증

4. **EINIT 실행**: 하드웨어 명령어를 통한 Enclave 활성화

이 과정에서 URTS는 복잡한 메모리 관리와 보안 검증을 수행하며, CLoader 클래스는 ELF 바이너리를 SGX 하드웨어가 이해할 수 있는 형태로 변환하는 핵심 역할을 담당합니다. 전체 과정은 하드웨어와 시뮬레이션 모드에서 동일한 인터페이스를 제공하여 개발자에게 투명한 환경을 제공합니다.